IA Aplicada com LLMs

Aula 15 — A conta, e o que quebra

Roteiro

1. **A cobrança das medições em aberto** — e onde o custo mora
2. **As alavancas** — roteamento, lote, tática zero
3. **A conferência da promessa** — o fecho do arco do trabalho
4. **O que quebra depois de subir** — cinco dívidas, duas pagas com a suíte de *evals*
5. **O que existe agora** — a entrega à aula de sistemas multiagente

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **consolidar** as medições de consumo das aulas anteriores numa conta por execução;
- **aplicar** as alavancas em ordem de retorno, **medindo** o efeito de cada uma;
- **fechar** a conferência da promessa da Parte 1;
- **resolver** as dívidas que vencem em produção, usando a suíte de *evals* nas que dependem dela.

A cobrança das medições em aberto

Aula	O que foi medido	O que ficou em aberto
06	a assimetria de embutir	quanto custa reconstruir o índice a cada reinício
09	as duas versões	o consumo em execuções reais
10	a ocupação das declarações	quanto ela custa por execução

Este bloco fecha as três. E é o **primeiro momento do curso** em que o aluno vê quanto o próprio sistema custa por execução.

Onde o custo mora

```
UMA EXECUÇÃO (8 passos, 2 servidores MCP conectados)

  declarações de ferramenta (MCP) .......  6.400 tk    38%
  reenvio do histórico (arquitetura) ....  7.900 tk    47%
  retornos de ferramenta ................  2.100 tk    12%
  resposta final ........................  400 tk      2%
  ──────────────────────────────────────────────────────────
  total ..................................... 16.800 tk
  custo ..................................... R$ 0,041
```

Lido no painel, e não suposto.

As duas fontes que dominam

O reenvio do histórico. A aula de arquitetura de agentes fez a conta: a cada volta a trajetória
inteira é reenviada, e o acumulado cresce com o **quadrado** do número de passos. Trinta passos: mais de 250 mil tokens acumulados.

As declarações de ferramenta. A aula de MCP mediu: dezenas de milhares em toda volta, antes de qualquer mensagem.

E a terceira, frequente e menos óbvia: **o retorno de ferramenta não encurtado** — a tática zero de arquitetura de agentes.

Parte 1

As alavancas

Roteamento de modelo

Trabalhos de roteamento relatam preservar **~95% da qualidade** do modelo forte enviando a ele entre **14% e 26%** das chamadas (Ong et al., 2024).

É a regra da aula de escolha de modelos — *um modelo por trabalho* — convertida em dinheiro.

E tem um pré-requisito, que a aula de *evals* forneceu:

> **Sem suíte, não há como saber se o modelo barato basta** para uma classe de casos. A suíte, rodada com o barato e lida por fatia, decide o que pode ser roteado.

As três, comparadas

Alavanca	Redução típica	Preço
roteamento	75–85% no tráfego roteado	a suíte de *evals* valida o barato por fatia
lote	grande, no não interativo	latência
tática zero (arquitetura de agentes)	variável, **frequentemente a maior**	trabalho de projeto de ferramenta

A pergunta que decide o lote é simples: *alguém está esperando por isto agora?*

Parte 2

A conferência da promessa

O fecho do arco do trabalho

A Parte 1 exigiu a **justificativa de negócio**: por que agente e não software comum, que ganho se espera — com linha de base, conta à vista e denominador.

Naquele momento, o número era uma **promessa**.

Agora existe painel.

A conferência

```
PROMETIDO (Parte 1)   redução de 80% no tempo de análise
                      linha de base: 14 min por despesa

MEDIDO (Parte 3)      4,2 min por despesa  ->  redução de 70%
                      custo por execução: R$ 0,041
                      1.900 execuções/mês ->  R$ 78/mês

a diferença, explicada:
  - 12% das despesas caem em exceção e exigem revisão humana
    (não estavam na linha de base)
  - a partida a frio acrescenta 40 s às primeiras execuções do dia
```

E a lição que a disciplina quer deixar

Prometer 80% e entregar 30%, com a conta à vista, é resultado aceitável e honesto. E é o que acontece na maioria dos projetos reais.

Prometer "mais eficiência" e não ter como conferir, não é.

> O inaceitável não é errar a estimativa. É ter prometido algo que **não se pode conferir**.

Parte 3

O que quebra depois de subir

Cinco dívidas, e duas se pagam com a suíte

Prompt registry (*prompt engineering*) — mudar prompt sem *deploy* é a vantagem, e é a
mesma propriedade que faz a mudança escapar da revisão. **A imagem fica igual, a *tag* fica igual, o comportamento muda**, e o painel não mostra a diferença.

Depreciação de modelo (escolha de modelos) — o provedor anuncia, dá prazo, e depois a
chamada falha. **Sem instrumento, migrar é apostar** — e o prazo não negocia.

> As duas pedem o mesmo instrumento — **a suíte de *evals***:
> a política de que a mudança no *registry* passa pela suíte antes de valer, e a migração decidida com a suíte rodada sobre o modelo novo.

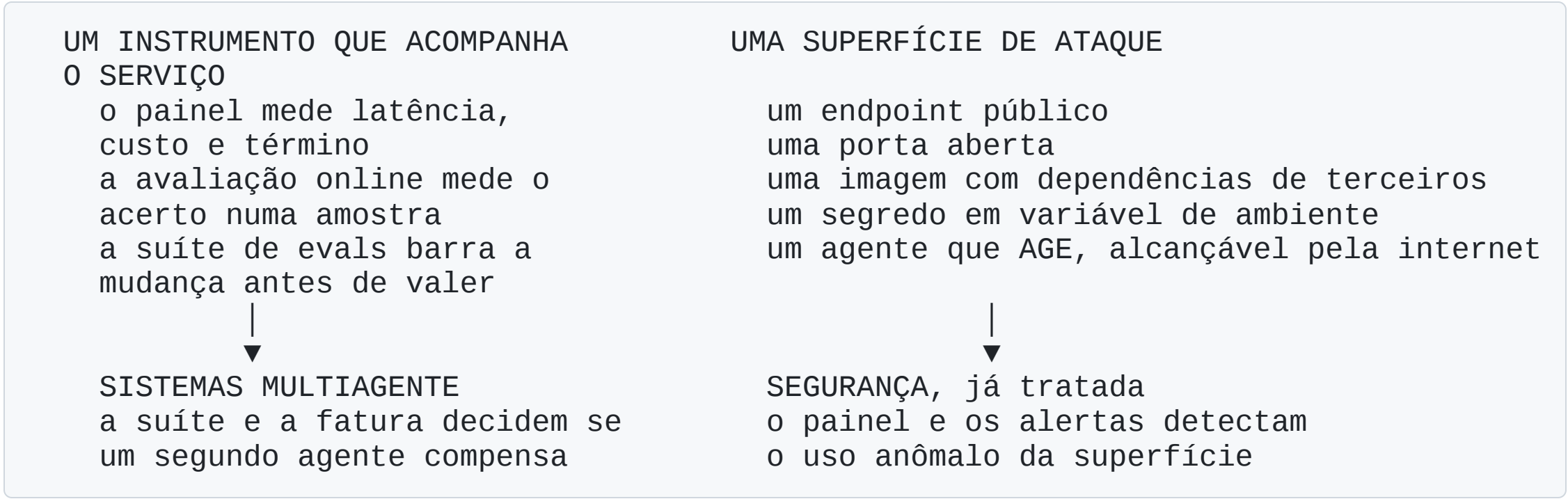
E três que se resolvem aqui

Memória sem teto (memória) — o crescimento é monotônico, e alguém paga armazenamento e latência de consulta. Exige política de retenção.

Versão do índice (*embeddings* e RAG) — reindexar com outro modelo de embedding muda o
`recall` **sem que ninguém tenha mudado o código**.

Custo que muda sem aviso — o provedor reajusta preços, e o modelo novo pode ser mais caro por token e mais barato por tarefa, ou o contrário. **A conta se refaz, não se herda.**

O que existe agora, e não existia



O carimbo, ao fim desta aula

Aula	Campos
03 · 05 · 06 · 07 · 08	prompt, modelo, parâmetros, arquitetura, *chunking*, `k`, prompt de resposta, memória
09 · 10	framework, especificação, servidor
11	versão do *dataset*, versão da rubrica do juiz
12	**versão implantada** — a *tag* da imagem

O painel precisa distinguir execuções de versões diferentes — sob pena de **somar sistemas distintos** num mesmo número.